



CS 465 Module Three Assignment Guidelines and Rubric

Overview

In this assignment, you will complete customer-facing webpages based on customer requirements provided in a wireframe document, adapting the document to the software constraints and MEAN stack architecture. You will take the static HTML site and move it to dynamic JSON using Handlebars (HBS).

Specifically, in this assignment, you will use Visual Studio (VS) Code to edit the HBS templates and JavaScript within the Express framework to implement Model View Controller (MVC)-style functionality.

Prompt

Continue developing the full stack web application for Travlr Getaways. Follow the steps in the Module Three Full Stack Guide linked in the Supporting Materials section.

Specifically, you must address the following rubric criteria:

- Develop and run a customer-facing front-end **architecture** that meets client software requirements.
- Dynamically **render** webpages by using the templating engine HBS.
- Use **routes** and views to align with the client's software requirements using MVC.
- **Test** to ensure that static HTML transitions to dynamic JSON data and renders properly.

What to Submit

Submit a zipped file folder, travlr.zip, that includes the updated Travlr website. The trips.json file should be included as part of the zipped file folder.

Note: You will push your local Git repository module3 branch to your GitHub repository. This procedure will become second nature to you as you move through the course modules. In the end, you will have a full stack web application with branches that represent key stages of building a full stack application leading up to full project completion.

Supporting Materials

The following resource supports your work on the assignment:

Resource: [Module Three Full Stack Guide](#)

Follow the instructions in this guide to complete the assignment and support your development of the full stack web application.

Module Three Assignment Rubric

Criteria	Meets Expectations (100%)	Partially Meets Expectations (70%)	Does Not Meet Expectations (0%)	Value
Architecture	Develops and runs a customer-facing front-end architecture that meets client software requirements	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include addressing code errors	Does not attempt criterion	25
Views: Rendering	Dynamically renders webpages by using a templating engine	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include the webpages' ability to dynamically render data	Does not attempt criterion	30
Routes: MVC Controllers	Uses routes and views to align with the client's software requirements using MVC	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include using partials or view fragments in common UI elements	Does not attempt criterion	30
Testing	Ensures that static HTML transitions to dynamic JSON data and renders properly	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include data not displaying correctly or rendering problems	Does not attempt criterion	15
Total:				100%